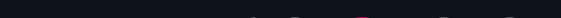
Anton Johansson

anjo@rev.ng

September 5 – KVM Forum 2025

Research paid for by Qualcomm Innovation Center, Inc.

Machine readable ISA $\rightarrow$ QEMU + tests



- rev.ng Labs, based in Milan
- Building a decompiler based on QEMU and LLVM
- Consultancy on compilers, emulators, and dynamic binary translation

Recap - helper-to-tcg

Xqci & Xqccmp

RISC-V Unified DB (UDB)

From UDB to QEMU

Testing

Outlook & Summary

Sequel to our previous talk..

Anton Johansson
anjo@rev.ng

Alessandro Di Federico
ale@rev.ng

June 15 – KVM Forum 2023

Research paid for by Qualcomm Innovation Center, Inc.

Helper:

```
int32_t A2_paddt(int32_t dst,
                 int32_t pred,
                 int32_t src1,
                 int32_t src2)
{
    if (pred & 1) {
        dst = src1 + src2;
    }
    return dst;
}
```

Helper:

```
int32_t A2_paddt(int32_t dst,
                 int32_t pred,
                 int32_t src1,
                 int32_t src2)
{
    if (pred & 1) {
        dst = src1 + src2;
    }
    return dst;
}
```

Tiny Code Generators (TCG):

```
void A2_paddt(TCGv_i32 dst,
              TCGv_i32 pred
              TCGv_i32 src1,
              TCGv_i32 src2)
{
    TCGv_i32 and = tcg_temp_new_i32();
    tcg_gen_andi_i32(and, pred, 1);

    TCGv_i32 add = tcg_temp_new_i32();
    tcg_gen_add_i32(add_1, src1, src2);

    tcg_gen_movcond_i32(TCG_COND_EQ, dst,
                pred, tcg_constant_i32(0),
                dst, add);
}
```

```
if (p0) r3 = add(r1,r2)
```

```
if (p0) r3 = add(r1,r2)
```

```
if (p0) r3 = add(r1,r2)
```

Helper:

```
mov     0x8(%rbp),%esi
mov     %esi,0x128(%rbp)
mov     $0x3,%r9d
mov     %eax,%r8d
mov     0xc(%rbp),%ecx
mov     0x100(%rbp),%edx
mov     %rbp,%rdi
call    *0xb4(%rip)
mov     %eax,0x128(%rbp)
mov     %eax,0x8(%rbp)
```


rewng

```
if (p0) r3 = add(r1,r2)
```

Helper:

```
mov     0x8(%rbp),%esi
mov     %esi,0x128(%rbp)
mov     $0x3,%r9d
mov     %eax,%r8d
mov     0xc(%rbp),%ecx
mov     0x100(%rbp),%edx
mov     %rbp,%rdi
call    *0xb4(%rip)
mov     %eax,0x128(%rbp)
mov     %eax,0x8(%rbp)
```

```
if (p0) r3 = add(r1,r2)
```

Helper:

```
mov     0x8(%rbp),%esi
mov     %esi,0x128(%rbp)
mov     $0x3,%r9d
mov     %eax,%r8d
mov     0xc(%rbp),%ecx
mov     0x100(%rbp),%edx
mov     %rbp,%rdi
call    *0xb4(%rip)
    <A2_paddt>:
        add     %r8d,%ecx
        mov     %esi,%eax
        and     $0x1,%edx
        cmovne  %ecx,%eax
        ret
mov     %eax,0x128(%rbp)
mov     %eax,0x8(%rbp)
```


rewng

```
if (p0) r3 = add(r1,r2)
```

Helper:

```
mov     0x8(%rbp),%esi
mov     %esi,0x128(%rbp)
mov     $0x3,%r9d
mov     %eax,%r8d
mov     0xc(%rbp),%ecx
mov     0x100(%rbp),%edx
mov     %rbp,%rdi
call    *0xb4(%rip)
    <A2_paddt>:
        add     %r8d,%ecx
        mov     %esi,%eax
        and     $0x1,%edx
        cmovne  %ecx,%eax
        ret
mov     %eax,0x128(%rbp)
mov     %eax,0x8(%rbp)
```

```
if (p0) r3 = add(r1,r2)
```

Helper:

```
mov     0x8(%rbp),%esi
mov     %esi,0x128(%rbp)
mov     $0x3,%r9d
mov     %eax,%r8d
mov     0xc(%rbp),%ecx
mov     0x100(%rbp),%edx
mov     %rbp,%rdi
call    *0xb4(%rip)
    <A2_paddt>:
        add     %r8d,%ecx
        mov     %esi,%eax
        and     $0x1,%edx
        cmovne  %ecx,%eax
        ret
mov     %eax,0x128(%rbp)
mov     %eax,0x8(%rbp)
```

TCG:

```
mov     0x8(%rbp),%r15d
mov     0xc(%rbp),%ebx
mov     0x100(%rbp),%r11d
and     $0x1,%r11d
add     %r15d,%ebx
cmp     %r13d,%r11d
cmovne  %ebx,%r10d
mov     %r10d,0x128(%rbp)
mov     %r10d,0x8(%rbp)
```


rewng

```
if (p0) r3 = add(r1,r2)
```

Helper:

```
mov     0x8(%rbp),%esi
mov     %esi,0x128(%rbp)
mov     $0x3,%r9d
mov     %eax,%r8d
mov     0xc(%rbp),%ecx
mov     0x100(%rbp),%edx
mov     %rbp,%rdi
call    *0xb4(%rip)
    <A2_paddt>:
        add     %r8d,%ecx
        mov     %esi,%eax
        and     $0x1,%edx
        cmovne  %ecx,%eax
        ret
mov     %eax,0x128(%rbp)
mov     %eax,0x8(%rbp)
```

TCG:

```
mov     0x8(%rbp),%r15d
mov     0xc(%rbp),%ebx
mov     0x100(%rbp),%r11d
and     $0x1,%r11d
add     %r15d,%ebx
cmp     %r13d,%r11d
cmovne  %ebx,%r10d
mov     %r10d,0x128(%rbp)
mov     %r10d,0x8(%rbp)
```


rev.ng

```
if (p0) r3 = add(r1,r2)
```

Helper:

```
mov     0x8(%rbp),%esi
mov     %esi,0x128(%rbp)
mov     $0x3,%r9d
mov     %eax,%r8d
mov     0xc(%rbp),%ecx
mov     0x100(%rbp),%edx
mov     %rbp,%rdi
call    *0xb4(%rip)
    <A2_paddt>:
        add     %r8d,%ecx
        mov     %esi,%eax
        and     $0x1,%edx
        cmovne  %ecx,%eax
        ret
mov     %eax,0x128(%rbp)
mov     %eax,0x8(%rbp)
```

TCG:

```
mov     0x8(%rbp),%r15d
mov     0xc(%rbp),%ebx
mov     0x100(%rbp),%r11d
and     $0x1,%r11d
add     %r15d,%ebx
cmp     %r13d,%r11d
cmovne  %ebx,%r10d
mov     %r10d,0x128(%rbp)
mov     %r10d,0x8(%rbp)
```


rewng

```
r1 = #1
r2 = #2
p0 = #0xff
if (p0) r3 = add(r1,r2)
```


rewng

```
r1 = #1
r2 = #2
p0 = #0xff
if (p0) r3 = add(r1,r2)
```

Helper:

```
mov     0x8(%rbp),%esi
mov     %esi,0x128(%rbp)
mov     $0x3,%r9d
mov     %eax,%r8d
mov     0xc(%rbp),%ecx
mov     0x100(%rbp),%edx
mov     %rbp,%rdi
call    *0xb4(%rip)
    <A2_paddt>:
        add     %r8d,%ecx
        mov     %esi,%eax
        and     $0x1,%edx
        cmovne  %ecx,%eax
        ret
mov     %eax,0x128(%rbp)
mov     %eax,0x8(%rbp)
```


rev.ng

```
r1 = #1
r2 = #2
p0 = #0xff
if (p0) r3 = add(r1,r2)
```

Helper:

```
mov     0x8(%rbp),%esi
mov     %esi,0x128(%rbp)
mov     $0x3,%r9d
mov     %eax,%r8d
mov     0xc(%rbp),%ecx
mov     0x100(%rbp),%edx
mov     %rbp,%rdi
call    *0xb4(%rip)
    <A2_paddt>:
        add     %r8d,%ecx
        mov     %esi,%eax
        and     $0x1,%edx
        cmovne  %ecx,%eax
        ret
mov     %eax,0x128(%rbp)
mov     %eax,0x8(%rbp)
```

TCG:

```
movl    $0x3,0x128(%rbp)
movl    $0x3,0x8(%rbp)
```


rewng

Helper:

TCG:

Helper:

- ✓ Easy to implement
- × No cross-call-site optimization

TCG:

revng

Helper:

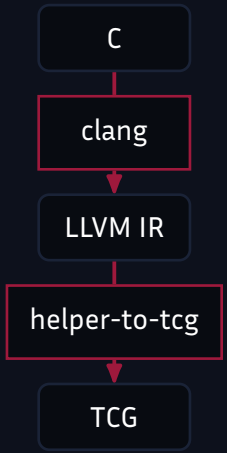
- ✓ Easy to implement
- × No cross-call-site optimization

TCG:

- ✓ Constant propagation
- ✓ Dead code elimination
- × Tedious, error prone

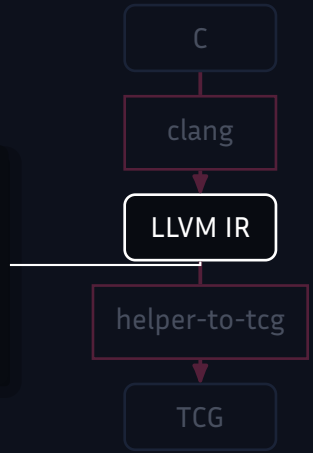
C

rewng

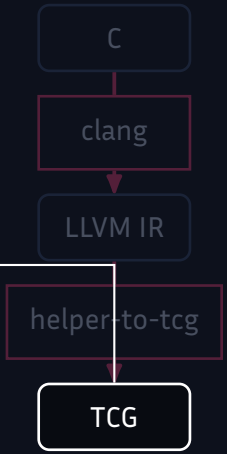

C
clang
LLVM IR
helper-to-tcg
TCG

```c
uint32_t A2_paddt(uint32_t dest,
                  uint32_t cond,
                  uint32_t a,
                  uint32_t b) {
    if (cond & 1) {
        dest = a + b;
    }
    return dest;
}
```

```
define i32 @A2_paddt(i32 %0, i32 %1,
                     i32 %2, i32 %3) {
  %5 = and i32 %1, 1
  %6 = icmp eq i32 %5, 0
  %7 = add i32 %3, %2
  %8 = select i1 %6, i32 %0, i32 %7
  ret i32 %8
}
```



```
void A2_paddt(TCGv_i32 dest_out, TCGv_i32 dest_in,
              TCGv_i32 cond, TCGv_i32 a, TCGv_i32 b) {
    TCGv_i32 temp2 = tcg_temp_new_i32();
    tcg_gen_andi_i32(temp2, cond, 1);

    TCGv_i32 temp5 = tcg_temp_new_i32();
    tcg_gen_add_i32(temp5, b, a);

    tcg_gen_movcond_i32(TCG_COND_EQ, dest_out, temp2,
        tcg_constant_i32(0), dest_in, temp5);
}
```



- Build-time tool translating LLVM IR $\rightarrow$ TCG
- Applied to Hexagon
  - Supports 1400/2000 instructions
- Upstreaming effort
  - Subproject
  - RFC last year

Recap - helper-to-tcg

Xqci & Xqccmp

RISC-V Unified DB (UDB)

From UDB to QEMU

Testing

Outlook & Summary

- Qualcomms RISC-V extensions
  - Goal: Improving instruction density
  - Xqci
    - 16-, 32-, 48-bit ISA
    - Saturating arithmetic, wide immediates, …
  - Xqccmp
    - Variation on Zcmp
    - Stack frames, double moves
  - Xqci 0.1 release Aug. 2024
    - We got involved v0.2 in September
    - No toolchain support

- Emulator-the-loop extension design
  1. Rapid prototyping + design-space exploration
  2. Cross-version validation
  3. Immediate toolchain testing
- Goal: Produce and test a QEMU version for each version of the Xqci/Xqccmp specification

Recap - helper-to-tcg

Xqci & Xqccmp

RISC-V Unified DB (UDB)

From UDB to QEMU

Testing

Outlook & Summary

- Single source of truth for RISC-V extensions
  - Machine readable
  - Definitions of: instructions, CSRs, extensions
- Backends for:
  - Specifications
  - Verifiers
  - Instruction Set Simulator
- Project headed by Derek Hower (Qualcomm), Afonso Oliviera (Synopsys)

- For more
  - Fosdem 2025, *RISC-V Unified Database: Streamlining the Ecosystem with a Centralized Source of Truth*, A. Oliviera; https://fosdem.org/2025/schedule/event/fosdem-2025-.../
  - https://github.com/riscv-software-src/riscv-unified-db

```
kind:
name:
long_name:
description:
definedBy:
base:
encoding:
assembly:
access:
operation():
```

```yaml
kind: instruction
name: qc.c.muliadd
long_name: Multiply and accumulate (Immediate)
description: |
  Increments `rd` by the multiplication of `rs1` and an unsigned immediate
  Instruction encoded in CL instruction format.
definedBy:
  anyOf:
    - Xqci
    - Xqciac
base: 32
...
```

```
...
encoding:
  match: 001----------10
  variables:
    - name: uimm
      location: 5|12-10|6
    - name: rs1
      location: 9-7
    - name: rd
      location: 4-2
...
```

```
...
assembly: " xd, xs1, uimm"
access:
  s: always
  u: always
  vs: always
  vu: always
...
```

```
...
operation(): |
  XReg reg = creg2reg(rd);
  XReg src = creg2reg(rs1);
  XReg orig_val = X[reg];
  X[reg] = orig_val + (X[src] * uimm);
```

Recap - helper-to-tcg

Xqci & Xqccmp

RISC-V Unified DB (UDB)

From UDB to QEMU

Testing

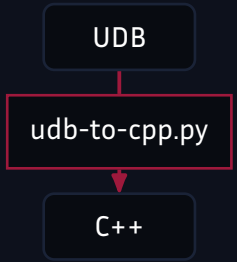
Outlook & Summary

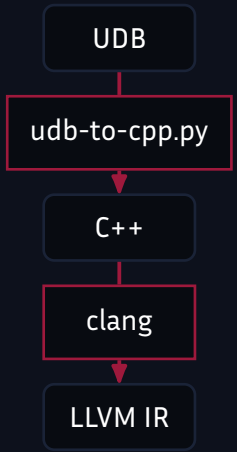
revng

- Need to handle
  - Instruction definitions (TCG)
  - Decode and disassembly
  - Access and definitions of Control Status Registers (CSRs)

```
XReg reg = creg2reg(rd);
XReg src = creg2reg(rs1);
XReg orig_val = X[reg];
X[reg] = orig_val + (X[src] * uimm);
```

UDB

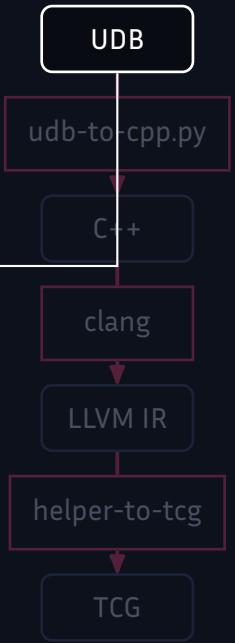

UDB
udb-to-cpp.py
C++



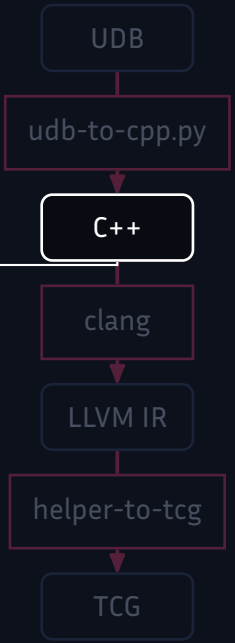
rev.ng


UDB
udb-to-cpp.py
C++
clang
LLVM IR
helper-to-tcg
TCG

```
XReg reg = creg2reg(rd);
XReg src = creg2reg(rs1);
XReg orig_val = X[reg];
X[reg] = orig_val + (X[src] * uimm);
```



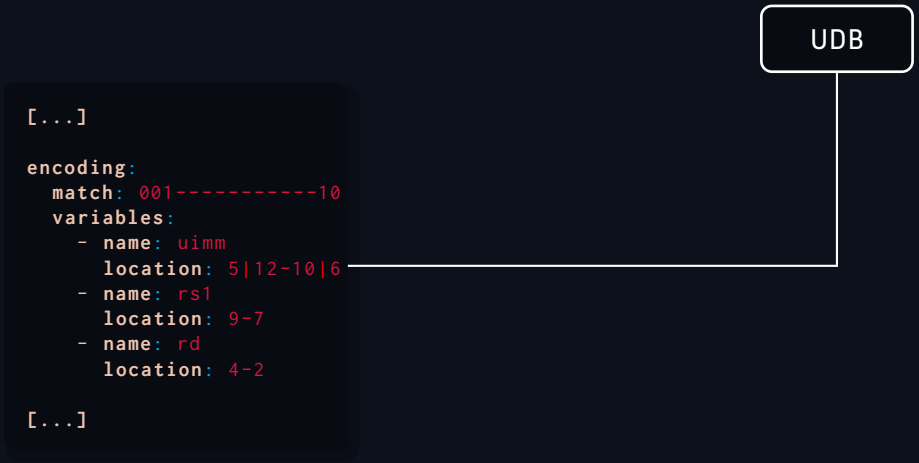
```
void qc_c_muliadd(Bits<5> uimm,
                  uint8_t rs1,
                  uint8_t rd) {
   XReg reg = creg2reg(rd);
   XReg src = creg2reg(rs1);
   XReg orig_val = X[reg];
   xqci_set_gpr_xreg(reg, orig_val + (X[src] * uimm));
}
```

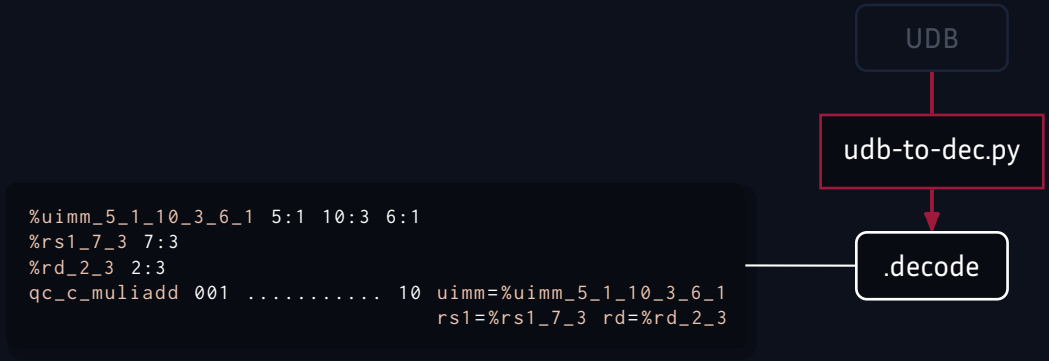


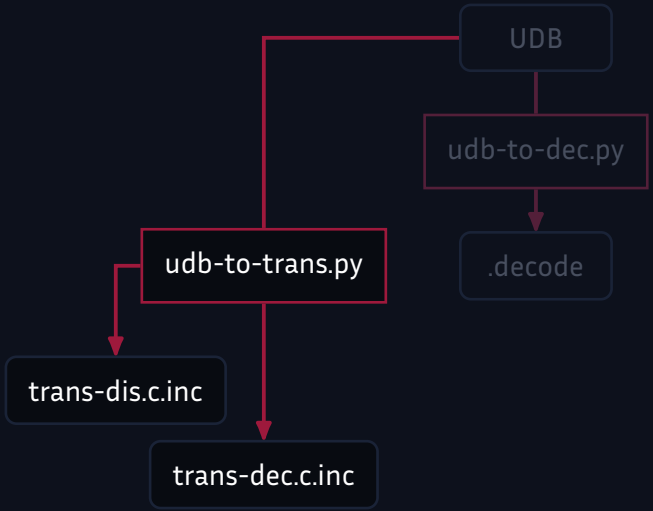
```
define void @qc_c_muliadd(ptr %0, i8 %1, i8 %2, i8 %3) {
  %5 = and i8 %3, 7
  %6 = or i8 %5, 8
  %7 = zext i8 %6 to i32
  %8 = and i8 %2, 7
  %9 = or i8 %8, 8
  %10 = zext i8 %9 to i32
  %11 = call i32 @xqci_get_gpr(i32 %7)
  %12 = call i32 @xqci_get_gpr(i32 %10)
  %13 = and i8 %1, 31
  %14 = zext i8 %13 to i32
  %15 = mul i32 %12, %14
  %16 = add i32 %15, %11
  %17 = zext i8 %6 to i64
  %18 = getelementptr %struct.CPUArchState, ptr %0, i64 0, i32 0,
                                            i32 0, i64 %17, i32 0
  store i32 %16, ptr %18
  ret void
}
```

```
void emit_qc_c_muliadd(DisasContext *ctx, TCGv_env env,
                       uint8_t vi_8, uint8_t vi_14,
                       uint8_t vi_17) {
    TCGv_i32 temp5 = xqci_get_gpr(ctx, ((vi_17 & 7) | 8));
    TCGv_i32 temp6 = xqci_get_gpr(ctx, ((vi_14 & 7) | 8));
    TCGv_i32 temp0 = tcg_temp_new_i32();
    tcg_gen_muli_i32(temp0, temp6, (vi_8 & 31));
    tcg_gen_add_i32(temp0, temp0, temp5);
    tcg_gen_mov_i32(cpu_gpr[((vi_17 & 7) | 8)], temp0);
}
```

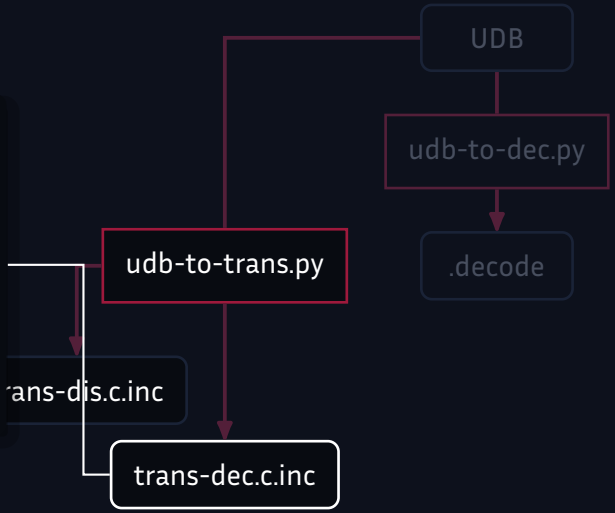
- Need to handle
  - ✓ Instruction definitions (TCG)
  - Decode and disassembly
  - Access and definitions of Control Status Registers (CSRs)


UDB
[...]
encoding:
match: 001----------10
variables:
- name: uimm
location: 5|12-10|6
- name: rs1
location: 9-7
- name: rd
location: 4-2
[...]

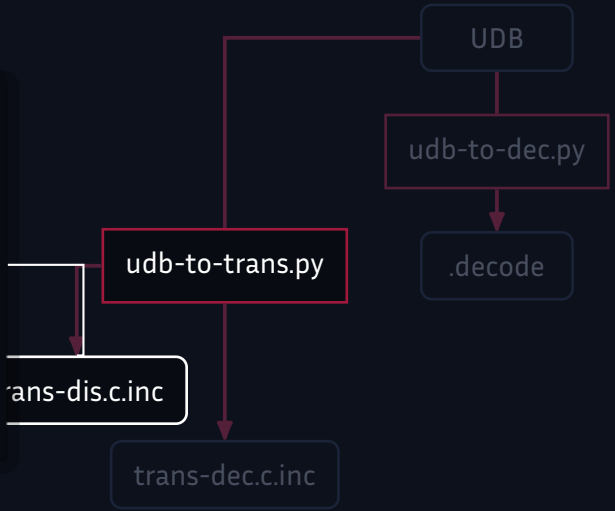

UDB
udb-to-dec.py
.decode
%uimm_5_1_10_3_6_1 5:1 10:3 6:1
%rs1_7_3 7:3
%rd_2_3 2:3
qc_c_muliadd 001 ..... 10 uimm=%uimm_5_1_10_3_6_1
rs1=%rs1_7_3 rd=%rd_2_3


UDB
udb-to-dec.py
.decode
udb-to-trans.py
trans-dis.c.inc
trans-dec.c.inc

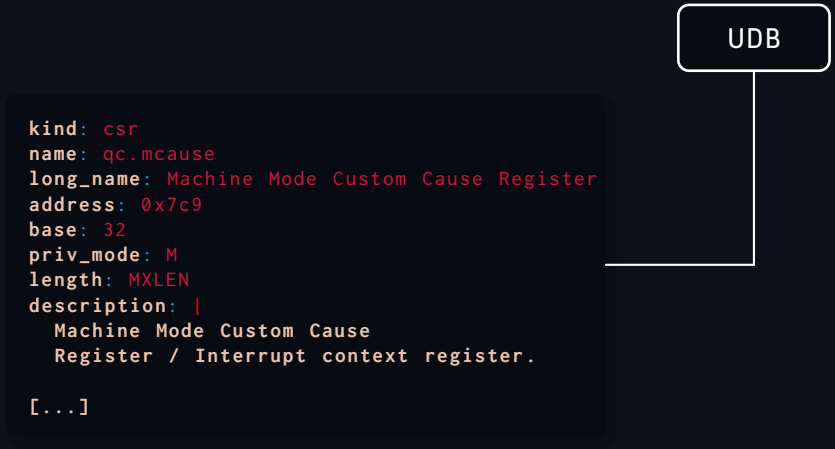
```
bool
trans_qc_c_muliadd(DisasContext *ctx,
                   arg_qc_c_muliadd *arg)
{
    if (!ctx->cfg_ptr->ext_xqci &&
        !ctx->cfg_ptr->ext_xqciac) {
        return false;
    }
    emit_qc_c_muliadd(ctx, tcg_env,
       arg->uimm, arg->rs1, arg->rd);
    return true;
}
```



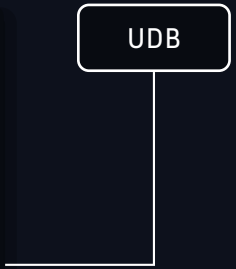
```
bool
trans_qc_c_muliadd(rv_decode *dec,
            arg_qc_c_muliadd *arg)
{
    if (!dec->cfg->ext_xqci &&
        !dec->cfg->ext_xqciac) {
        return false;
    }
    dec->uimm = arg->uimm;
    dec->rs1 = arg->rs1;
    dec->rd = arg->rd;
    dec->op = rv_op_qc_c_muliadd;
    return true;
}
```



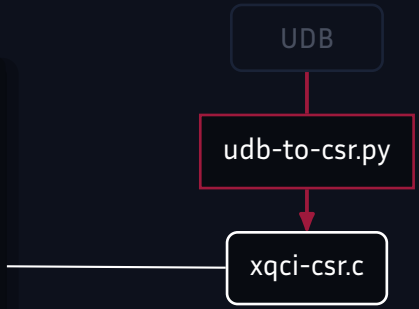
- Need to handle
  - ✓ Instruction definitions (TCG)
  - ✓ Decode and disassembly
  - Access and definitions of Control Status Registers (CSRs)


UDB
kind: csr
name: qc.mcause
long_name: Machine Mode Custom Cause Register
address: 0x7c9
base: 32
priv_mode: M
length: MXLEN
description: |
Machine Mode Custom Cause
Register / Interrupt context register.
[...]

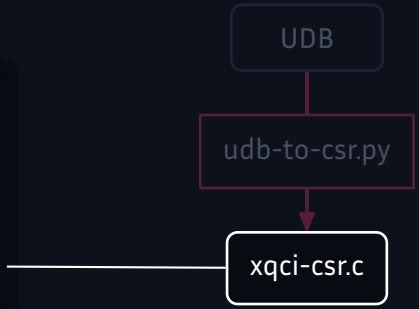
```
[...]

definedBy:
  anyOf:
    - name: Xqci
      version: ">=0.7"
    - name: Xqciint
      version: ">=0.4"
fields:
  INT:
    location: 31
    description: Alias of mcause.INT bit, if 1'b1 - ...
    type: RW-H
    reset_value: 0
  NMIE:
    location: 30
    description: Alias of mnstatus.NMIE, if 1'b0, ...
    type: RW-H
    reset_value: 0
```



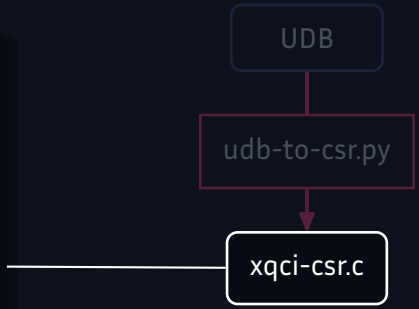
```
#define CSR_QC_MCAUSE 0x7c9
#define QC_MCAUSE_INT 0x80000000
#define QC_MCAUSE_NMIE 0x40000000
#define QC_MCAUSE_MPDT 0x20000000
#define QC_MCAUSE_MNPIE 0x10000000
#define QC_MCAUSE_MPIE 0x8000000
#define QC_MCAUSE_MIE 0x4000000
#define QC_MCAUSE_EXCP 0x2000000
#define QC_MCAUSE_NMIP 0x1000000
#define QC_MCAUSE_MNPIL 0xf00000
#define QC_MCAUSE_MPIL 0xf0000
#define QC_MCAUSE_MIL 0xf000
#define QC_MCAUSE_EXCCODE 0xfff

[...]
```



```
[...]

RISCVException pred_qc_mcause(CPURISCVState *env,
                              int index,
                              uint64_t bit)
{
    if (env->priv != PRV_M ||
        !riscv_cpu_cfg(env)->ext_xqciint &&
        !riscv_cpu_cfg(env)->ext_xqci) {
        return RISCV_EXCP_ILLEGAL_INST;
    }
    return RISCV_EXCP_NONE;
}

[...]
```



```
[...]

RISCVException rmw_qc_mcause(CPURISCVState *env,
                             int csrno,
                             target_ulong *ret_val,
                             target_ulong new_val,
                             target_ulong wr_mask)
{
    if (ret_val) {
        *ret_val = env->qc_mcause;
    }
    env->qc_mcause = (env->qc_mcause & ~wr_mask) |
                     (new_val & wr_mask);
    return RISCV_EXCP_NONE;
}

[...]
```



- Need to handle
  - ✓ Instruction definitions (TCG)
  - ✓ Decode and disassembly
  - ✓ Access and definition of Control Status Registers (CSRs)

- CSRs with custom read/write logic
- Allowing helper-to-tcg to use declared functions

```
void emit_qc_c_muliadd(DisasContext *ctx, TCGv_env env,
                       uint8_t vi_8, uint8_t vi_14,
                       uint8_t vi_17) {
    TCGv_i32 temp5 = xqci_get_gpr(ctx, ((vi_17 & 7) | 8));
    TCGv_i32 temp6 = xqci_get_gpr(ctx, ((vi_14 & 7) | 8));
    TCGv_i32 temp0 = tcg_temp_new_i32();
    tcg_gen_muli_i32(temp0, temp6, (vi_8 & 31));
    tcg_gen_add_i32(temp0, temp0, temp5);
    tcg_gen_mov_i32(cpu_gpr[((vi_17 & 7) | 8)], temp0);
}
```

- CSRs with custom read/write logic
- Allowing helper-to-tcg to use declared functions

```
void emit_qc_c_muliadd(DisasContext *ctx, TCGv_env env,
                       uint8_t vi_8, uint8_t vi_14,
                       uint8_t vi_17) {
    TCGv_i32 temp5 = xqci_get_gpr(ctx, ((vi_17 & 7) | 8));
    TCGv_i32 temp6 = xqci_get_gpr(ctx, ((vi_14 & 7) | 8));
    TCGv_i32 temp0 = tcg_temp_new_i32();
    tcg_gen_muli_i32(temp0, temp6, (vi_8 & 31));
    tcg_gen_add_i32(temp0, temp0, temp5);
    tcg_gen_mov_i32(cpu_gpr[((vi_17 & 7) | 8)], temp0);
}
```

- Modifications to the RISC-V frontend
  - Support for 16-, and 48-bit dynamic decoders
  - Fields in CPU state for CSRs
  - Glue code is autogenerated
- Xqci inst. support
  - 16-bit: 22/22
  - 32-bit: 110/110
  - 48-bit: 25/25
- Xqccmp inst. support
  - 16-bit: 7/7

Recap - helper-to-tcg

Xqci & Xqccmp

RISC-V Unified DB (UDB)

From UDB to QEMU

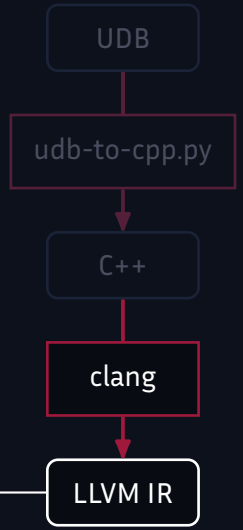
Testing

Outlook & Summary

- Two approaches
  - Single-instruction tests
  - Existing test suites

revng

```
define void @qc_c_muliadd(ptr %0, i8 %1, i8 %2, i8 %3) {
  %5 = and i8 %3, 7
  %6 = or i8 %5, 8
  %7 = zext i8 %6 to i32
  %8 = and i8 %2, 7
  %9 = or i8 %8, 8
  %10 = zext i8 %9 to i32
  %11 = call i32 @xqci_get_gpr(i32 %7)
  %12 = call i32 @xqci_get_gpr(i32 %10)
  %13 = and i8 %1, 31
  %14 = zext i8 %13 to i32
  %15 = mul i32 %12, %14
  %16 = add i32 %15, %11
  %17 = zext i8 %6 to i64
  %18 = getelementptr %struct.CPUArchState, ptr %0, i64 0, i32 0,
                                            i32 0, i64 %17, i32 0

  store i32 %16, ptr %18
  ret void
}
```



rev.ng

- KLEE
  1. Symbolic execution engine on LLVM IR
  2. Collects conditions on branch $\Rightarrow$ SAT solver
  3. https://klee-se.org/

C code

```c
int main() {
    uint32_t x;
    klee_make_symbolic(&x, sizeof(x), "x");
    if (x > 16) {
        printf("First branch!\n");
    } else {
        printf("Second branch!\n");
    }
    return 0;
}
```


rev.ng

C code

```c
int main() {
    uint32_t x;
    klee_make_symbolic(&x, sizeof(x), "x");
    if (x > 16) {
        printf("First branch!\n");
    } else {
        printf("Second branch!\n");
    }
    return 0;
}
```

KLEE output

```
test1
    x: 0

test2
    x: 24
```

C code

```c
int main() {
    uint32_t x;
    klee_make_symbolic(&x, sizeof(x), "x");
    if (x > 16) {
        printf("First branch!\n");
    } else {
        printf("Second branch!\n");
    }
    return 0;
}
```

KLEE output

```
test1
    x: 0

test2
    x: 24
```

Output

```
test1
    Second branch

test2
    First branch
```

C code

```c
uint32_t add(uint32_t a, uint32_t b) {
    return a + b;
}

int main() {
    uint32_t a;
    uint32_t b;
    klee_make_symbolic(&a, sizeof(a), "a");
    klee_make_symbolic(&b, sizeof(b), "b");
    uint32_t c = add(a, b);
    printf("c: %d\n", c);
    return 0;
}
```


rev.ng

C code

```c
uint32_t add(uint32_t a, uint32_t b) {
    return a + b;
}

int main() {
    uint32_t a;
    uint32_t b;
    klee_make_symbolic(&a, sizeof(a), "a");
    klee_make_symbolic(&b, sizeof(b), "b");
    uint32_t c = add(a, b);
    printf("c: %d\n", c);
    return 0;
}
```

KLEE output

```
test1
    a: 0
    b: 0
```

C code

```c
uint32_t add(uint32_t a, uint32_t b) {
    return a + b;
}

int main() {
    uint32_t a;
    uint32_t b;
    klee_make_symbolic(&a, sizeof(a), "a");
    klee_make_symbolic(&b, sizeof(b), "b");
    uint32_t c = add(a, b);
    printf("c: %d\n", c);
    return 0;
}
```

KLEE output

```
test1
    a: 0
    b: 0
```

Output

```
test1
    c: 0
```

C code

```c
bool has_overflow = false;

uint32_t add(uint32_t a, uint32_t b) {
    if ((uint64_t) a + (uint64_t) b > UINT32_MAX) {
        has_overflow = true;
    }
    return a + b;
}

int main() {
    uint32_t a = 0;
    uint32_t b = 0;
    klee_make_symbolic(&a, sizeof(a), "a");
    klee_make_symbolic(&b, sizeof(b), "b");
    uint32_t c = add(a, b);
    printf("c: %u\n", c);
    printf("has_overflow: %u\n", has_overflow);
    return 0;
}
```


rev.ng

C code

```c
bool has_overflow = false;

uint32_t add(uint32_t a, uint32_t b) {
    if ((uint64_t) a + (uint64_t) b > UINT32_MAX) {
        has_overflow = true;
    }
    return a + b;
}

int main() {
    uint32_t a = 0;
    uint32_t b = 0;
    klee_make_symbolic(&a, sizeof(a), "a");
    klee_make_symbolic(&b, sizeof(b), "b");
    uint32_t c = add(a, b);
    printf("c: %u\n", c);
    printf("has_overflow: %u\n", has_overflow);
    return 0;
}
```

KLEE output

```
test1
    a: 0
    b: 0

test2
    a: 4228907007
    b: 4228907007
```


rev.ng

C code

```c
bool has_overflow = false;

uint32_t add(uint32_t a, uint32_t b) {
    if ((uint64_t) a + (uint64_t) b > UINT32_MAX) {
        has_overflow = true;
    }
    return a + b;
}

int main() {
    uint32_t a = 0;
    uint32_t b = 0;
    klee_make_symbolic(&a, sizeof(a), "a");
    klee_make_symbolic(&b, sizeof(b), "b");
    uint32_t c = add(a, b);
    printf("c: %u\n", c);
    printf("has_overflow: %u\n", has_overflow);
    return 0;
}
```

KLEE output

```
test1
    a: 0
    b: 0

test2
    a: 4228907007
    b: 4228907007
```

Output

```
test1
    c: 0
    has_overflow: 0

test2
    c: 4162846718
    has_overflow: 1
```


rev.ng

- Repeat for other arithmetic operations
- Loads/Stores
  - Address in controlled region
  - Keep track of values being loaded/stored
- Jumps
  - Controlled destination address

rev.ng

- KLEE output can be used to generate test binaries for edge cases!
  - Raw ELF binaries
  - C tests w. inline assembly (toolchain)
- Xqci tests
  - Raw: 293
  - C: 117
- Xqccmp tests
  - Raw: 42
  - C: 2

- Two approaches
  - ✓ Single-instruction tests
  - Existing test suites

- Requires toolchain support
- SPEC
- embench
- picolibc (in progress)
- $\Rightarrow$ Uncovers discrepencies between compiler and spec in UDB

- Two approaches
  - ✓ Single-instruction tests
  - ✓ Existing test suites

Recap - helper-to-tcg

Xqci & Xqccmp

RISC-V Unified DB (UDB)

From UDB to QEMU

Testing

Outlook & Summary

- Machine readable ISA $\rightarrow$ QEMU + tests
- Emulator early in design process useful for
  - Catching bugs early
  - Iterating on instructions

- Generate tests for other extensions?
- Test system mode thoroughly
  - Semihosted picolibc tests currently fail
  - CSR read/write testing with KLEE
- Upstreaming
  - Two main dependencies: helper-to-tcg, UDB
  - Both as subprojects and make extension optional?
  - Commits cleaned up generated code?
  - …?

Get in touch at:
info@rev.ng

Subscribe to our newsletter:
https://rev.ng/newsletter-subscribe